

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по рубежному контролю №2
Вариант Г21

Выполнил:
Студент группы ИУ5-35Б
Перфильев С.М.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

ЗАДАНИЕ

- 1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.
- 2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

ЛИСТИНГ ПРОГРАММЫ

main.py

```
import unittest
from operator import itemgetter

class Operator:
    def __init__(self, id, name, usage, lang_id):
        self.id = id
        self.name = name
        self.usage = usage
        self.lang_id = lang_id

class Lang:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class OperLang:
    def __init__(self, lang_id, oper_id):
        self.lang_id = lang_id
        self.oper_id = oper_id

langs = [
    Lang(1, 'Ассемблер'),
    Lang(2, 'Алгол'),
    Lang(3, 'Питон'),
    Lang(4, 'Java'),
    Lang(5, 'Ада'),
]

opers = [
    Operator(1, 'MOV', 100, 1),
```

```

    Operator(2, 'ADD', 80, 1),
    Operator(3, 'IF', 120, 3),
    Operator(4, 'FOR', 110, 3),
    Operator(5, 'BEGIN', 90, 2),
    Operator(6, 'END', 70, 2),
    Operator(7, 'LOOP', 60, 5),
]

opers_langs = [
    OperLang(1, 1),
    OperLang(1, 2),
    OperLang(2, 5),
    OperLang(2, 6),
    OperLang(3, 3),
    OperLang(3, 4),
    OperLang(3, 6),
    OperLang(5, 7),
]

def get_one_to_many(langs, opers):
    return [
        (op.name, op.usage, lang.name)
        for lang in langs
        for op in opers
        if op.lang_id == lang.id
    ]

def get_many_to_many(langs, opers_langs, opers):
    many_to_many_temp = [
        (lang.name, ol.lang_id, ol.oper_id)
        for lang in langs
        for ol in opers_langs
        if lang.id == ol.lang_id
    ]
    return [
        (op.name, op.usage, lang_name)
        for lang_name, lang_id, oper_id in many_to_many_temp
        for op in opers if op.id == oper_id
    ]

def task_g1(langs, one_to_many):

```

```

res_g1 = {}
for lang in langs:
    if lang.name.startswith('A'):
        lang_ops = list(filter(lambda x: x[2] == lang.name, one_to_many))
        op_names = [name for name, _, _ in lang_ops]
        res_g1[lang.name] = op_names
return res_g1

def task_g2(langs, one_to_many):
    res_g2_unsorted = []
    for lang in langs:
        lang_ops = list(filter(lambda x: x[2] == lang.name, one_to_many))
        if lang_ops:
            usages = [usage for _, usage, _ in lang_ops]
            max_usage = max(usages)
            res_g2_unsorted.append((lang.name, max_usage))
    return sorted(res_g2_unsorted, key=itemgetter(1), reverse=True)

def task_g3(many_to_many):
    return sorted(many_to_many, key=itemgetter(2))

class TestTasks(unittest.TestCase):
    def setUp(self):
        self.one_to_many = get_one_to_many(langs, opers)
        self.many_to_many = get_many_to_many(langs, opers_langs, opers)

    def test_task_g1(self):
        expected = {
            'Ассемблер': ['MOV', 'ADD'],
            'Алгол': ['BEGIN', 'END'],
            'Ада': ['LOOP']
        }
        self.assertEqual(task_g1(langs, self.one_to_many), expected)

    def test_task_g2(self):
        expected = [
            ('Питон', 120),
            ('Ассемблер', 100),
            ('Алгол', 90),
            ('Ада', 60)
        ]

```

```
self.assertEqual(task_g2(langs, self.one_to_many), expected)

def test_task_g3(self):
    expected = [
        ('LOOP', 60, 'Ада'),
        ('BEGIN', 90, 'Алгол'),
        ('END', 70, 'Алгол'),
        ('MOV', 100, 'Ассемблер'),
        ('ADD', 80, 'Ассемблер'),
        ('IF', 120, 'Питон'),
        ('FOR', 110, 'Питон'),
        ('END', 70, 'Питон')
    ]
    self.assertEqual(task_g3(self.many_to_many), expected)

if __name__ == '__main__':
    unittest.main(verbosity=2)
```

СКРИНШОТЫ РАБОТЫ ПРОГРАММЫ

```
(lab_env) PS C:\Users\Stepa\PyCharmMiscProject\rk2> python main.py
test_task_g1 (__main__.TestTasks.test_task_g1) ... ok
test_task_g2 (__main__.TestTasks.test_task_g2) ... ok
test_task_g3 (__main__.TestTasks.test_task_g3) ... ok
```

```
-----
Ran 3 tests in 0.001s
```

```
OK
```

```
(lab_env) PS C:\Users\Stepa\PyCharmMiscProject\rk2> python -m unittest main.py
...
```

```
-----
Ran 3 tests in 0.001s
```

```
OK
```